# 产品需求文档 (PRD)：职通车（MVP版）

| | |
|---|---|
| 项目名称 | 职业通车 |
| 版本号 | v1.0 (Hackathon MVP) |
| 产品形态 | Web 端 SaaS 工具 (PC 桌面版) |
| 核心价值 | 将非结构化招聘流程转化为“原子级”可计算流水线，降低小公司错配成本。 |
| 目标用户 | 早期创业公司 CEO/CTO、无专职 HR 的小团队负责人。 |

## 1. 产品概述

### 1.1 背景与痛点

小公司招聘难的本质不是缺乏简历，而是信噪比低和处理摩擦大。

- 输入模糊：JD（职位描述）写不清楚，导致源头错配。
- 黑盒匹配：传统筛选靠“感觉”，缺乏可验证的证据链。
- 执行繁琐：筛选后的邀约/拒信环节存在大量重复劳动。

### 1.2 产品定位

不是一个流量招聘平台（如Boss直聘），而是一个本地化的招聘效能工具。

它是一个“流水线工厂”：

输入（模糊需求 + 批量简历 ） -> AI原子化处理 -> 输出（排序列表 + 证据链 + 自动邮件）。

### 1.3 MVP 核心目标

- 时间：从上传简历到发出面试邀请，耗时缩短至 10 分钟以内。
- 质量：Top 3 推荐候选人的面试通过率 > 50%。
- 体验：实现“零手动输入”，全流程点击/确认即可。

## 2. 核心功能流程

设计原则：尽量减少用户输入，AI 负责生成，人负责确认。

1. **[启动]** 用户输入一句话招聘需求（e.g., “招一个懂React的前端实习生”）。
2. **[澄清]** AI 扮演面试官，反向提问 5-8 个关键问题（澄清JD）。
3. **[结构化]** 系统生成结构化 JD，用户确认。
4. **[导入]** 用户批量上传简历文件（PDF/png）。
5. **[计算]** 系统后台进行简历解析（OCR/Text）与原子级匹配。
6. **[决策]** 展示带有“证据链”的候选人排序列表。
7. **[执行]** 用户勾选心仪候选人，一键生成并发送面试邀请邮件。

## 3. 功能详细需求

### 模块一：输入与澄清

对应痛点**1**：需求不清晰导致匹配偏差。

| 功能点 | 详细描述 | 优先级 | 开发建议 |
|---|---|---|---|
| 对话式**JD**生成 | 用户输入模糊需求，AI (LLM) 启动多轮对话。AI需引导用户明确：核心技能(Must-have)、经验年限、软技能、加分项。 | **P0** | Prompt工程：设定System Prompt为“苛刻的招聘专家”。 |
| 结构化**Schema**预览 | 对话结束后，将自然语言转化为 JSON 格式的“岗位画像”。<br><br>字段包括：Role, Hard_Skills, Exp_Years, Education, Soft_Skills。 | **P0** | 前端需可视化展示Key-Value 对，允许用户手动微调。 |

### 模块二：简历解析与匹配

对应痛点**2 & 3**：非结构化数据与证据链缺失。

| 功能点 | 详细描述 | 优先级 | 开发建议 |
| --- | --- | --- | --- |
| 批量简历解析 | 支持拖拽上传 PDF/png 文件（上限 50 份）。后台利用 OCR/LLM 提取简历关键字段，对应 JD 的 Schema。 | **P0** | 使用开源解析库 + LLM 进行纠错。 |
| 原子级匹配打分 | 基于“规则 + LLM”混合评分。<br><br>1. 硬性过滤（规则）：学历/年限不符直接 Fail。<br><br>2. 语义匹配（LLM）：技能匹配度打分 (0-100)。 | **P0** | 关键点：不仅给分，要给出 Reasoning。 |
| 证据链提取 | 在展示结果时，必须引用简历原文。例如：“匹配度高 - 候选人项目经历中明确提到了‘使用 Redux 优化性能’，符合 JD 要求。” | **P0** | 这是产品的“灵魂”，解决信任问题。 |

## 模块三：决策与输出

对应痛点**4**：执行摩擦大。

| 功能点 | 详细描述 | 优先级 | 开发建议 |
| --- | --- | --- | --- |
| 候选人看板 | 列表展示：姓名 | 匹配分 | 核心优势 Sum |
| 一键自动化 | 提供两个按钮： | **P0** | 预设高情商邮件模板，允许用户再次编 |

| | | | |
|---|---|---|---|
| | 1. **Book Meeting**：生成个性化面试邀请邮件（自动填入候选人名字、岗位、日历链接）。<br><br>2. **Reject**：生成委婉的拒信。 | | 辑。 |
| **ROI** 仪表盘 | (加分项) 简单的统计：本次筛选节省了多少时间，过滤了多少不合格简历。 | P1 | 简单的数字展示即可。 |

# 4. 技术架构与落地

鉴于比赛时间短，不追求高并发和复杂后端。

- 前端 **(Web做MVP):**
  - 框架：React 。
  - 交互：重点打磨“Chat Interface”（JD生成）和“List View”（匹配结果）。
- 后端 **:**
  - 语言：Python (FastAPI 或 Flask)。
  - 核心逻辑：LangChain (用于编排 LLM 调用)。
- **AI** 模型**:**
  - LLM：调用魔搭或自行搭建工作流（coze、影刀）。
- 数据存储**:**
  - 无需复杂数据库，使用本地 Session 或 SQLite 临时存储。

# 5. 风险与规避

| 潜在风险 | 对应原子真理 | 规避方案 **(Hackathon**策略**)** |
|---|---|---|
| **LLM** 幻觉 | 风险：匹配不准 | **Evidence-Based:** 强制模型输出引用原文，若找不到原文则不给分。 |
| **Token** 超限 | 风险：成本/速度 | 分段处理**:** 先提取简历摘要， |

| | | |
|---|---|---|
| | | 再进行匹配；不要把整个 PDF 丢给模型对话。 |
| 简历格式怪异 | 风险：解析失败 | 降级策略：如果解析失败，标记为“需人工查看”，不强行处理，保证系统不崩。 |

## 6. 加分项

如果核心流程（上传 -> 匹配 -> 发邮件）跑通了，还有时间，可以做以下功能作为亮点：

1. **AI** 面试题生成：在生成面试邀请邮件的同时，根据该候选人的简历短板，给面试官生成 3 道定制化面试题（e.g., “该候选人项目经验丰富但缺乏基础算法描述，建议问：XXX”）。
2. 多语言支持：如果是跨国招聘，支持英文 JD 匹配中文简历（利用 LLM 的翻译能力）。

## 7. 界面原型建议

- **Page 1 - Landing:** 极简风格。中间一个大对话框：“你想招什么人？”
- **Page 2 - Chat:** 左侧是聊天窗口，右侧实时生成 JD 的卡片。
- **Page 3 - Dashboard:** 顶部：本次任务摘要（岗位 + 简历数量）。
  - 主体：候选人列表。每一行点击展开，左边是简历原文高亮，右边是 AI 的评价和证据。
  - 底部：Floating Action Button -> "Generate Emails".

### 备注

不要做注册登录，不要做用户中心。

为了演示流畅，直接进入主功能。

每次使用都是一次性的 Session，用完即走，这符合“工具”而非“平台”的定位，也能大幅减少开发工作量。